

ISW-521 - AMBIENTE WEB I

React

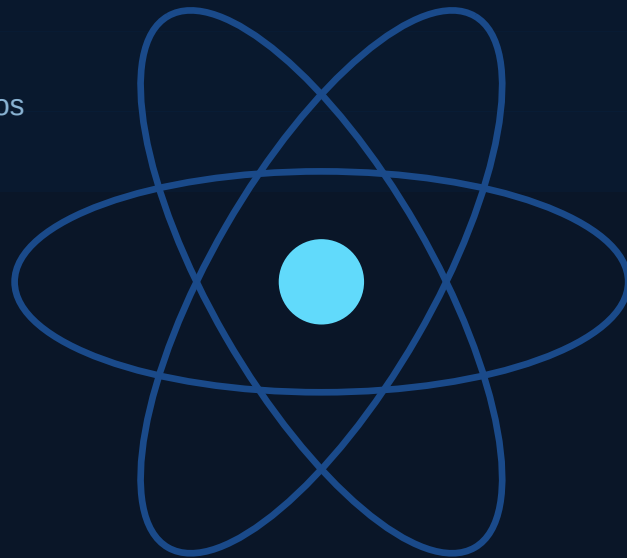
De 0 a 100

Arquitectura - Funcionamiento interno - Comparativas - Ecosistema 2026

ESTUDIANTE

Byron Bolanos Zamora

Codigo: ISW-521 - Ambiente Web I - UTN Sede San Carlos



CONTENIDO

Tabla de Contenidos

1	Definición técnica — qué es React y qué problema resuelve	3
2	Arquitectura interna — cómo funciona React por dentro	5
3	Cuándo usar React (y cuándo no)	7
4	Comparativa: React vs. Svelte	9
5	Ecosistema actual — adopción, versión y tendencias 2026	11
6	Referencias	13

1 Definición técnica

¿Qué es React?

React es una **biblioteca de JavaScript de código abierto** desarrollada por Meta (Facebook), diseñada específicamente para construir interfaces de usuario (UI) declarativas y basadas en componentes. No es un framework: React no impone una arquitectura completa de aplicación ni incluye enrutamiento, manejo de formularios ni capas de red. Su alcance está intencionalmente acotado a una sola responsabilidad: **describir cómo debería verse la UI en cualquier estado dado** y actualizarla eficientemente cuando ese estado cambia.

■ DEFINICIÓN PRECISA — NO FRAMEWORK

React es una **biblioteca de UI**, no un framework. Un framework (como Angular o Next.js) incluye decisiones de arquitectura completas (router, HTTP, inyección de dependencias). React solo resuelve el problema del renderizado y la reactividad. El resto del stack lo elige el desarrollador. Esta distinción es técnica y tiene implicaciones reales en cómo se compone y despliega una aplicación.

El problema concreto que resuelve

En 2011, el equipo de Facebook enfrentaba un problema de ingeniería específico: la interfaz del chat y el contador de notificaciones se desincronizaban con frecuencia. El código que manipulaba el DOM directamente se volvía frágil porque **múltiples partes de la UI dependían del mismo dato pero lo actualizaban por rutas distintas**. El modelo imperativo del DOM (`getElementById`, `innerHTML`, `addEventListener`) no tenía un mecanismo para garantizar consistencia cuando el estado cambiaba.

El modelo mental de React

La solución de React es cambiar el paradigma de *imperativo* a *declarativo*. En vez de decirle al navegador **qué hacer paso a paso** ("encontrá este nodo, cambiale este atributo"), el desarrollador describe **cómo debería verse la UI** para un estado determinado, y React se encarga de la transición.

$$UI = f(state)$$

La UI es una función pura del estado. Dado el mismo estado, siempre produce el mismo resultado.

Qué problema NO resuelve React

React no resuelve: enrutamiento de páginas, fetch de datos, caché, autenticación, validación de formularios, internacionalización, ni accesibilidad avanzada de forma nativa. Todos estos problemas requieren bibliotecas externas o frameworks sobre React (como Next.js, React Router, TanStack Query, React Hook Form, etc.).

Breve historia y contexto

2011	Problema real	Facebook detecta desincronización UI en chat/notificaciones.
2011–13	Creación	Jordan Walke crea FaxJS, inspirado en XHP (PHP). Primer uso interno en Facebook.
Mayo 2013	Open Source	Presentado en JSConf US. La comunidad lo recibe con escepticismo por JSX.
Ene 2015	React Native	React se extiende a apps nativas iOS/Android con la misma mentalidad de componentes.
Feb 2019	React Hooks	useState/useEffect reemplazan clases. Cambio de paradigma radical en la comunidad.
Mar 2022	React 18	Concurrent rendering, Suspense estable, startTransition.
2023–25	React 19	Server Components estables, use() hook, acciones del servidor, compilador React Compiler.

2 Arquitectura interna

¿Cómo funciona React por dentro?

React opera en varias capas bien definidas. Entender cada capa permite escribir código más eficiente, diagnosticar problemas de performance y anticipar el comportamiento del framework en escenarios complejos.

1. JSX y la transformación a JavaScript

JSX es azúcar sintáctica: no es HTML ni una extensión del navegador. El compilador (Babel, SWC o el nuevo React Compiler) transforma cada elemento JSX en una llamada a **React.createElement()**, que retorna un objeto JavaScript plano llamado *React element* (nodo del Virtual DOM).

```
JSX → JavaScript (compilación)

// JSX que el desarrollador escribe
const el = <Button color="blue">Guardar</Button>;

// Lo que el compilador genera
const el = React.createElement(Button, { color: "blue" }, "Guardar");

// El objeto React element resultante (Virtual DOM node)
// { type: Button, props: { color: "blue", children: "Guardar" }, key: null }
```

2. Virtual DOM y el algoritmo de Reconciliación (Diffing)

El **Virtual DOM** es una representación en memoria del árbol de la UI, implementada como objetos JavaScript planos. React mantiene dos copias: el árbol actual y el árbol pendiente (trabajo en progreso). Cuando el estado cambia:

- React genera un **nuevo árbol Virtual DOM** llamando a las funciones de componente afectadas.
- El algoritmo de **reconciliación (diffing)** compara el nuevo árbol contra el anterior. React usa una heurística $O(n)$ en vez de $O(n^3)$ (la complejidad óptima de comparar árboles genéricos): asume que nodos del mismo tipo producen estructuras similares, y usa la prop **key** para identificar elementos en listas.
- Solo los nodos que cambiaron se propagan al **renderer** (en web: react-dom) para actualizar el DOM real. Este paso se llama **commit**.

■ LA PROP KEY — POR QUÉ IMPORTA

Cuando React hace diffing de listas, usa la prop **key** para identificar qué elemento es cuál entre renders. Sin key (o con el índice del array como key), React puede reutilizar el nodo DOM incorrecto, causando bugs sutiles de estado: inputs con el valor equivocado, animaciones en el elemento incorrecto, pérdida de foco. La key debe ser un identificador estable y único del dato (un ID de base de datos, un slug, etc.), nunca el índice del array si la lista puede reordenarse o filtrarse.

3. Fiber — el motor de reconciliación

Desde React 16 (2017), el algoritmo de reconciliación se llama **Fiber**. La innovación clave: Fiber convierte el proceso de renderizado en una estructura de datos interrumpible. Antes (React <16), el render era una llamada recursiva síncrona que bloqueaba el hilo principal hasta terminar. Fiber divide el trabajo en unidades pequeñas ("fibers") que pueden pausarse, priorizarse y reanudarse.

Esto habilita el **Concurrent Rendering** (React 18+): React puede interrumpir un render de baja prioridad para procesar una interacción urgente (como un keypress), y luego retomar el render original. El desarrollador controla prioridades con **startTransition()** y el hook **useDeferredValue()**.

4. Flujo de datos: unidireccional

React impone un flujo de datos estrictamente **unidireccional (top-down)**. Un componente padre puede pasar datos a hijos via props, pero un hijo nunca modifica directamente el estado del padre. Para comunicación hijo→padre, el padre pasa una función callback como prop. Este modelo hace el flujo de datos predecible y fácil de trazar.



5. Hooks — el modelo de estado moderno

Introducidos en React 16.8 (2019), los Hooks son funciones que permiten a los componentes funcionales "engancharse" al ciclo de vida y estado de React, funcionalidad que antes solo estaba disponible en clases.

Hook	Propósito	Cuándo usarlo
useState()	Estado local del componente	Valores que cambian y deben re-renderizar
useEffect()	Efectos secundarios	Fetch, timers, suscripciones, manipulación del DOM

useRef()	Referencia mutable sin re-render	Acceder al DOM, almacenar valores entre renders
useContext()	Consumir contexto global	Datos que muchos componentes necesitan (tema, usuario)
useMemo()	Memorizar cálculos costosos	Evitar recalcular valores derivados en cada render
useCallback()	Memorizar funciones	Estabilizar callbacks para evitar re-renders de hijos
useReducer()	Estado complejo	Lógica de estado con múltiples sub-valores o transiciones

3

Cuándo usar React (y cuándo no)

Escenarios donde React es la elección correcta

- **SPAs con estado complejo:** Aplicaciones donde múltiples partes de la UI reaccionan al mismo estado (dashboards, editores, apps de tiempo real). El modelo de componentes + estado centralizado brilla aquí.
- **Equipos grandes o proyectos de larga duración:** La arquitectura de componentes fuerza separación de responsabilidades. Combinado con TypeScript, el código es más mantenible y escalable que con manipulación directa del DOM.
- **Ecosistema y contratación:** React tiene el mayor ecosistema de bibliotecas de UI, la mayor comunidad y el mayor mercado laboral. Para proyectos empresariales esto reduce riesgo operativo significativamente.
- **Apps multiplataforma:** React Native permite compartir lógica de negocio y estado entre web e iOS/Android. El mismo modelo mental reduce la curva de aprendizaje.
- **Renderizado del servidor (SSR/SSG):** Con Next.js, React permite generar HTML en el servidor para SEO y performance (Time to First Byte), con hidratación client-side para interactividad.

Escenarios donde React NO es la herramienta correcta

■ REACT TIENE UN COSTO BASE

React 19 pesa ~45 KB gzipped (react + react-dom). Para sitios de contenido estático o con muy poca interactividad, este overhead es injustificable. Un usuario en una conexión 3G lenta descargará, parseará y ejecutará ese bundle antes de ver contenido interactivo.

- **Sitios mayormente estáticos o de contenido:** Un blog, un sitio corporativo o una landing page con poca interactividad se sirven mejor con HTML/CSS puro, un generador estático (Astro, Hugo, Eleventy) o incluso una pizca de JavaScript vanilla.
- **Aplicaciones con restricciones de performance extremas:** Para interfaces de altísima frecuencia de actualización (trading, juegos, editores de audio), el overhead de Virtual DOM puede ser un cuello de botella. Svelte o manipulación directa del DOM puede ser más apropiado.
- **Proyectos pequeños de un solo desarrollador sin dependencias de equipo:** Vue o Svelte tienen curvas de aprendizaje más amables y menos boilerplate para prototipos o proyectos pequeños.

- **Entornos donde JavaScript está restringido o es indeseable:** Aplicaciones que requieren SSR completo sin hidratación client-side, o que deben funcionar sin JavaScript, se sirven mejor con arquitecturas server-side (Rails, Laravel, HTMX).

Tabla de decisión rápida

Escenario	¿Usar React?	Alternativa si no
SPA compleja con mucho estado	✓ Sí	—
Dashboard / app interna empresarial	✓ Sí	—
App con SSR y SEO crítico	✓ Sí (Next.js)	—
App móvil nativa	✓ Sí (RN)	Flutter, Swift, Kotlin
Blog / sitio de contenido	✗ No	Astro, Hugo, WordPress
Landing page simple	✗ No	HTML + CSS + JS vanilla
Prototipo rápido individual	■ Quizás	Vue, Svelte
UI con actualizaciones de 60fps+	■ Cuidado	Svelte, canvas directo
Sin JavaScript en cliente	✗ No	HTMX, Rails, Laravel

4 Comparativa: React vs. Svelte

La comparativa más instructiva para React en 2026 es **Svelte**: representa la filosofía opuesta en casi todas las dimensiones técnicas importantes. Svelte es el "anti-React" conceptual. Entender sus diferencias aclara qué decisiones de diseño tomó React y por qué.

Diferencia fundamental de arquitectura

React es una biblioteca de runtime: el código de React se envía al navegador y se ejecuta ahí. El Virtual DOM, la reconciliación y el sistema de componentes corren en el cliente en cada sesión de usuario.

Svelte es un compilador: en el momento del build, Svelte convierte los componentes .svelte en JavaScript imperativo altamente optimizado. No hay Virtual DOM, no hay runtime de Svelte en el navegador. El resultado son instrucciones directas de manipulación del DOM.

```
Mismo componente: React vs. Svelte

// React (JSX + useState)
import { useState } from "react";
function Contador() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(c => c + 1)}>{count}</button>;
}

<!-- Svelte (archivo .svelte) -->
<script>
  let count = 0;
</script>
<button on:click={() => count++}>{count}</button>
```

Comparativa técnica detallada

Dimensión	React 19	Svelte 5
Naturaleza	Biblioteca de runtime	Compilador (sin runtime)
Bundle size (hello world)	~45 KB gzip	~1–3 KB gzip
Virtual DOM	Sí (core del modelo)	No (compila a DOM directo)
Modelo de reactividad	Estado inmutable + hooks	Variables reactivas (\$state rune)
Lenguaje de plantillas	JSX (JS puro)	Sintaxis .svelte (HTML extendido)
TypeScript	Muy bueno (JSX tipado)	Excelente (nativo en .svelte)

Ecosistema	Enorme (npm, componentes)	Pequeño pero creciendo
Curva de aprendizaje	Moderada (hooks, JSX)	Baja (más intuitivo)
Adopción industrial	Dominante (82% State of JS)	Creciente (22% State of JS)
Server-side rendering	React Server Comp. (Next.js)	SvelteKit (oficial)
Comunidad/soporte	Masiva (Meta + comunidad)	Activa pero menor
Casos de uso ideales	Apps grandes, equipos, ecosistema	Performance crítica, proyectos pequeños

Ventajas y desventajas reales

✓ React — Ventajas

- Ecosistema masivo: millones de paquetes npm
- Mayor mercado laboral y comunidad
- Soporte corporativo estable (Meta)
- React Native para móvil nativo
- Server Components para SSR avanzado
- Flexibilidad total de stack

✗ React — Desventajas

- Bundle size base (~45 KB gzip)
- Curva de aprendizaje de hooks
- Requiere bibliotecas externas para todo
- Verbose comparado con Svelte
- Overhead de Virtual DOM en UIs simples
- Ecosistema fragmentado (muchas opciones)

✓ Svelte — Ventajas

- Bundle mínimo, sin runtime
- Sintaxis concisa y menos boilerplate
- Performance nativa de DOM directo
- Reactividad más intuitiva (variables)
- Excelente para proyectos medianos

✗ Svelte — Desventajas

- Ecosistema de componentes menor
- Menor adopción en empresas grandes
- Menos ofertas laborales
- Compilador como capa de abstracción
- Sin equivalente a React Native estable

■ CONCLUSIÓN DE LA COMPARATIVA

React gana en proyectos donde el ecosistema, la escala del equipo y la disponibilidad de talento son factores críticos. Svelte gana en performance de bundle, curva de aprendizaje y proyectos donde cada KB importa. Para ISW-521 y el contexto de desarrollo web profesional, React sigue siendo la elección más pragmática por su dominancia en la industria y su ecosistema.

5

Ecosistema actual 2026

Versión estable y estado de React en 2026

React 19 es la versión estable actual en producción (lanzada diciembre 2024). Las novedades más significativas frente a React 18 incluyen: el **React Compiler** (optimización automática sin useMemo/useCallback manual), **Server Components** estables (renderizado en servidor con acceso directo a bases de datos), el hook **use()** para consumir promesas y contexto de forma unificada, y las **Server Actions** para mutaciones de datos sin endpoint HTTP explícito.

Adopción en la industria (State of JS 2024 / npm trends)

Tecnología	Uso (State of JS 2024)	Descargas npm/semana (2026)
React	82%	~26 millones
Vue	49%	~4.8 millones
Angular	43%	~3.5 millones
Svelte	22%	~1.2 millones
Solid	10%	~340 mil

Fuentes: State of JS 2024 (stateofjs.com), npm trends (npmrends.com) — junio 2026.

Las empresas detrás del ecosistema

Meta (Facebook)	Mantenedora principal de React core
Vercel	Next.js (framework React más usado), React Server Components
Shopify	Remix (framework React), React Native Web
Microsoft	TypeScript (estándar de facto para React)
TanStack	TanStack Query, TanStack Router, TanStack Form
Radix UI / shadcn	Primitivos de UI y componentes sin estilo

El stack de React más usado en producción (2026)

Capa	Herramienta	Por qué
Framework	Next.js 15	SSR, SSG, App Router, Server Components, deploy en Vercel

Estilos	Tailwind CSS 4	Utility-first, integración perfecta con componentes
Componentes UI	shadcn/ui + Radix	Sin dependencia de runtime, accesible, personalizable
Estado global	Zustand o Jotai	Minimalistas, sin boilerplate, reemplazan Redux en apps medianas
Datos del servidor	TanStack Query v5	Cache, sync, invalidación, refetch automático
Formularios	React Hook Form	Performance (no re-renderiza), validación con Zod
Testing	Vitest + Testing Library	Rápido, nativo con Vite, API familiar
Lenguaje	TypeScript 5	Estándar de la industria, tipos para props y hooks
Deploy	Vercel / Cloudflare Pages	Edge functions, SSR distribuido, DX de primer nivel

Tendencias verificables en 2026

- **React Server Components (RSC):** El cambio más disruptivo desde los Hooks. Permite renderizar componentes en el servidor con acceso directo a bases de datos, sin enviar lógica al cliente. Adoptado por Next.js 13+ como default.
- **React Compiler:** Elimina la necesidad de useMemo/useCallback manual. El compilador optimiza automáticamente los re-renders. Disponible en React 19, consolidado en producción desde 2025–2026.
- **Decline de Redux:** Las descargas de Redux se han estancado mientras Zustand y Jotai crecen. El estado del servidor (TanStack Query) ha eliminado el caso de uso principal de Redux en muchas apps.
- **TypeScript como estándar:** Más del 80% de proyectos React nuevos usan TypeScript. Los tipos para props y hooks eliminan una clase entera de bugs.
- **Edge rendering:** Vercel, Cloudflare y AWS Amplify soportan ejecutar React en edge networks globales, reduciendo latencia de SSR a <50ms.

■ ESTADO ACTUAL EN UNA LÍNEA

React 19 es la versión estable. Next.js + Tailwind + TypeScript + TanStack Query es el stack más demandado en la industria en 2026. React domina con ~26 millones de descargas semanales en npm, el doble del total de sus competidores combinados.

6 Referencias

[1] Documentación oficial de React

Meta Open Source. (2025). React Documentation. <https://react.dev>

[2] State of JavaScript 2024

Greif, S., Burel, E., & et al. (2024). State of JavaScript 2024. <https://stateofjs.com/2024>

[3] npm trends — React, Vue, Angular, Svelte

npm, Inc. (2026). npm trends: react vs vue vs angular vs svelte. <https://npmtrends.com>

[4] React 19 Release Notes

Meta React Team. (Diciembre 2024). React v19. <https://react.dev/blog/2024/12/05/react-19>

[5] Svelte 5 Documentation

Harris, R. & Svelte community. (2024). Svelte 5 Docs. <https://svelte.dev/docs/svelte/overview>

[6] React Fiber Architecture

Chedreau, C. (2017). React Fiber Architecture (GitHub). <https://github.com/acdlite/react-fiber-architecture>

[7] React Compiler Beta

Meta React Compiler Team. (2024). Introducing the React Compiler.
<https://react.dev/learn/react-compiler>

[8] Next.js Documentation — App Router

Vercel. (2026). Next.js 15 Documentation. <https://nextjs.org/docs>

[9] Understanding Virtual DOM

Abramov, D. (2015). "React Components, Elements, and Instances". React Blog.
<https://legacy.reactjs.org/blog/2015/12/18/react-components-elements-and-instances.html>

[10] TanStack Query v5

Tanner Linsley. (2024). TanStack Query Documentation.
<https://tanstack.com/query/latest/docs/framework/react/overview>

Byron Bolaños Zamora

Universidad Técnica Nacional Sede San Carlos
Ingeniería en Desarrollo de Software Ambiente Web
I — ISW-521 · 2026